Effat University — College of Engineering

Department of Computer Science

CS3081 Artificial Intelligence

LAB 3: Propositional Logic & Model Checking

Group Challenge

Instructor	Dr. Naila Marir	**Duration**	1 hour 30 minutes
Semester	Fall 2026	**Group size**	3–4 students
Submission	One GitHub link per group	**Total marks**	100

Group No.			
Member name	**Student ID**	**Member name**	**Student ID**
Hamed Almehdhar	**S24109392**	Mohammad Tabahi	S24108951
Hasan Albaiti	**S23108331**		

4.Task1—UnderstandtheCode(30marks,20min)

1. `harry.py`: Truth table for `rain`, `hagrid`, `dumbledore` (8 rows)

Knowledge Base: `(¬rain ⇒ hagrid) ∧ (hagrid ∨ dumbledore) ∧ ¬(hagrid ∧ dumbledore) ∧ dumbledore`

Row	rain	hagrid	dumbledore	¬rain ⇒ hagrid	hagrid ∨ dumbledore	¬(hagrid ∧ dumbledore)	KB	Evaluation Notes
1	F	F	F	F	F	T	F	`¬rain ⇒ hagrid`, `hagrid ∨ dumbledore`, and `dumbledore` are False
2	F	F	T	F	T	T	F	`¬rain ⇒ hagrid` is False (since `¬rain` is True and `hagrid` is False)
3	F	T	F	T	T	T	F	Fact `dumbledore` is False
4	F	T	T	T	T	F	F	`¬(hagrid ∧ dumbledore)` is False (both cannot be True)
5	T	F	F	T	F	T	F	`hagrid ∨ dumbledore` and `dumbledore` are False
6 ✓	**T**	**F**	**T**	**T**	**T**	**T**	**T**	**Knowledge base is True (all four sentences hold)**
7	T	T	F	T	T	T	F	Fact `dumbledore` is False
8	T	T	T	T	T	F	F	`¬(hagrid ∧ dumbledore)` is False (both cannot be True)

Explanation of why the program prints `True`:

The KB is true in exactly one row (highlighted): rain = T, hagrid = F, dumbledore = T. The fact dumbledore forces dumbledore = T; then ¬(hagrid ∧ dumbledore) forces hagrid = F; then ¬rain ⇒ hagrid can only hold if ¬rain is false, i.e. rain = T. Because rain is true in every model where the KB is true, the KB entails rain, so model_check returns True.

Output (`harry.py`)

```
PS C:\Users\hamid\OneDrive\Desktop\CS3081\LABS\LAB 3> python harry.py
True
```

2. `clue.py`: Explain in plain English why `MsScarlet`, `library` and `knife` must be true

- MsScarlet: One of ColMustard, ProfPlum, MsScarlet must be the culprit. The facts rule out ColMustard (initial cards) and ProfPlum (known card), so only MsScarlet is left.
- library: One of ballroom, kitchen, library must be the room. Kitchen is ruled out by the initial cards and ballroom by a known card, so it must be the library.
- knife: One of knife, revolver, wrench must be the weapon. Revolver is ruled out by the initial cards. The unknown card says (¬MsScarlet v ¬library v ¬wrench); since MsScarlet and library are both true, wrench must be false. That leaves the knife.

Output (`clue.py`)

```
PS C:\Users\hamid\OneDrive\Desktop\CS3081\LABS\LAB 3> python clue.py
MsScarlet: YES
library: YES
knife: YES
```

5.Task2—ChangetheKnowledge(30marks,20min)

1. `harry.py`: Delete the last fact `dumbledore` from the knowledge base

- **(a) Prediction:** `False`.
- **(b) Result:** `False`.
- **(c) Explanation:** When `dumbledore` is removed from the knowledge base, Row 3 (`rain = F, hagrid = T, dumbledore = F`) also satisfies all remaining rules in the knowledge base, even though `rain` is False in that row. Because `rain` is no longer True in every model where the knowledge base is True, the knowledge base does not entail `rain`, and the program prints `False`.

Code (`harry.py`)

```
from logic import *

rain = Symbol("rain")
hagrid = Symbol("hagrid")
dumbledore = Symbol("dumbledore")

knowledge = And(
    Implication(Not(rain), hagrid),
    Or(hagrid, dumbledore),
    Not(And(hagrid, dumbledore))
)

print(model_check(knowledge,  rain))
```

Output (`harry.py`)

```
PS C:\Users\hamid\OneDrive\Desktop\CS3081\LABS\LAB 3> python harry.py
False
```

2. `puzzle.py`: Delete the clue `MinervaGryffindor`, then add one new clue of your own

- **(a) Prediction:** No one can be placed in a house.
- **(b) Result:** Nothing was printed.
- **(c) Explanation:** The remaining clues (Gilderoy is Gryffindor or Ravenclaw; Pomona is not Slytherin) leave 8 valid assignments of the four houses (for example Gilderoy–Gryffindor/Pomona–Hufflepuff/Minerva–Ravenclaw/Horace–Slytherin and Gilderoy–Ravenclaw/Pomona–Gryffindor/Minerva–Hufflepuff/Horace–Slytherin). No person–house pair is true in all 8, so no symbol is entailed and nothing is printed.

Output (After Deleting `MinervaGryffindor`)

```
PS C:\Users\hamid\OneDrive\Desktop\CS3081\LABS\LAB 3> python puzzle.py
PS C:\Users\hamid\OneDrive\Desktop\CS3081\LABS\LAB 3>
```

New Clue Added (`Symbol("HoraceRavenclaw")`): Adding the clue that Horace is in Ravenclaw makes the solution unique again: since Horace takes Ravenclaw and Gilderoy must be in Gryffindor or Ravenclaw, Gilderoy must be in **Gryffindor**. Only Hufflepuff and Slytherin remain for Pomona and Minerva. Since Pomona cannot be in Slytherin, Pomona is placed in **Hufflepuff**, leaving Minerva in **Slytherin**.

Code (`puzzle.py` Clue Change)

```
# Task 2.2: Deleted clue MinervaGryffindor
# knowledge.add(
#     Symbol("MinervaGryffindor")
# )

# Task 2.2: New clue that restores a unique house assignment
knowledge.add(
    Symbol("HoraceRavenclaw")
)
```

Output (With New Clue `HoraceRavenclaw`)

```
PS C:\Users\hamid\OneDrive\Desktop\CS3081\LABS\LAB 3> python puzzle.py
GilderoyGryffindor
PomonaHufflepuff
MinervaSlytherin
HoraceRavenclaw
```

6. Task 3 — Group Challenge: Knights & Knaves (30 marks, 30 min)

Code (`knights.py` — Puzzles 0, 1, 2, and Bonus Puzzle 3)

```
from logic import *

AKnight = Symbol("A is a Knight")
AKnave = Symbol("A is a Knave")

BKnight = Symbol("B is a Knight")
BKnave = Symbol("B is a Knave")

CKnight = Symbol("C is a Knight")
CKnave = Symbol("C is a Knave")


def basics(*people):
    """Every person is a knight or a knave, never both."""
    rules = And()
    for knight, knave in people:
        rules.add(Or(knight, knave))
        rules.add(Not(And(knight, knave)))
    return rules


def says(knight, knave, statement):
    """A knight's statement is true; a knave's statement is false."""
    return And(
        Implication(knight, statement),
        Implication(knave, Not(statement))
    )


# Puzzle 0: A says "I am both a knight and a knave."
knowledge0 = And(
    basics((AKnight, AKnave)),
    says(AKnight, AKnave, And(AKnight, AKnave))
)

# Puzzle 1: A says "We are both knaves." B says nothing.
knowledge1 = And(
    basics((AKnight, AKnave), (BKnight, BKnave)),
    says(AKnight, AKnave, And(AKnave, BKnave))
)

# Puzzle 2: A says "We are the same kind." B says "We are of different kinds."
same = Or(And(AKnight, BKnight), And(AKnave, BKnave))
different = Or(And(AKnight, BKnave), And(AKnave, BKnight))
knowledge2 = And(
    basics((AKnight, AKnave), (BKnight, BKnave)),
    says(AKnight, AKnave, same),
    says(BKnight, BKnave, different)
)

# Puzzle 3 (Bonus): A says either "I am a knight." or "I am a knave.",
```

```
# but we don't know which. B says "A said 'I am a knave'."
# B also says "C is a knave." C says "A is a knight."
#
# "A said X" is encoded as: A's saying X is consistent with A's type,
# i.e. (A is a knight) <=> X.
a_said_knight = Biconditional(AKnight, AKnight)   # X = "I am a knight"
a_said_knave = Biconditional(AKnight, AKnave)     # X = "I am a knave"
knowledge3 = And(
    basics((AKnight, AKnave), (BKnight, BKnave), (CKnight, CKnave)),
    Or(a_said_knight, a_said_knave),              # A said one of the two
    says(BKnight, BKnave, a_said_knave),          # B: "A said 'I am a knave'"
    says(BKnight, BKnave, CKnave),                # B: "C is a knave"
    says(CKnight, CKnave, AKnight)                # C: "A is a knight"
)


def main():
    symbols = [AKnight, AKnave, BKnight, BKnave, CKnight, CKnave]
    puzzles = [
        ("Puzzle 0", knowledge0),
        ("Puzzle 1", knowledge1),
        ("Puzzle 2", knowledge2),
        ("Puzzle 3 (Bonus)", knowledge3)
    ]
    for puzzle, knowledge in puzzles:
        print(puzzle)
        if len(knowledge.conjuncts) == 0:
            print("    Not yet implemented.")
        else:
            for symbol in symbols:
                if model_check(knowledge, symbol):
                    print(f"    {symbol}")


if __name__ == "__main__":
    main()
```

Output (`knights.py`)

```
PS C:\Users\hamid\OneDrive\Desktop\CS3081\LABS\LAB 3> python knights.py
Puzzle 0
    A is a Knave
Puzzle 1
    A is a Knave
    B is a Knight
Puzzle 2
    A is a Knave
    B is a Knight
Puzzle 3 (Bonus)
    A is a Knight
    B is a Knave
    C is a Knight
```

Explanations

- **1. Puzzle 0 (`A is a Knave`):** A says "I am both a knight and a knave." Because no person can be both at the same time, the statement is always false. Since knights always tell the truth, A cannot be a knight, so **A is a Knave**.
- **2. Puzzle 1 (`A is a Knave`, `B is a Knight`):** A says "We are both knaves." If A were a knight, the statement would have to be true, meaning A is a knave—a contradiction. Thus **A is a Knave**. Since A's statement is false and A is a knave, B cannot be a knave, so **B is a Knight**.
- **3. Puzzle 2 (`A is a Knave`, `B is a Knight`):** A says "We are the same kind" and B says "We are of different kinds." If A were a knight, they would be the same kind, making B a knight too—but then B's statement ("different kinds") would be false. Therefore **A is a Knave**, which means they are of different kinds and **B is a Knight**.

Bonus (+10 marks) — Puzzle 3 (`A is a Knight`, `B is a Knave`, `C is a Knight`): No islander can ever say "I am a knave" (a knight would never lie and say it, and a knave would never tell the truth and admit it). Therefore, B's claim that A said "I am a knave" must be false, proving **B is a Knave**. Because B always lies, B's second claim ("C is a knave") is also false, which means **C is a Knight**. Finally, since C tells the truth, C's statement ("A is a knight") is true, so **A is a Knight**.